

Tugas 2 - Struktur Sistem Operasi



Nama : Putra Ganda Prayoga

Nim : 2410651057

Prodi Informatika

Fakultas Teknik

Universitas Muhammadiyah Jember

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <unistd.h>

#define SHM_KEY 1234
#define SEM_KEY 5678
#define MAX_MSG 1024

// Struktur data untuk shared memory
struct chat_data {
    char messages[10][MAX_MSG];
    int count;
};

// Fungsi lock/unlock semaphore
void sem_lock(int semid) {
    struct sembuf sb = {0, -1, 0};
    semop(semid, &sb, 1);
}

void sem_unlock(int semid) {
    struct sembuf sb = {0, 1, 0};
    semop(semid, &sb, 1);
}

int main() {
    // Membuat shared memory dan semaphore
    int shmid = shmget(SHM_KEY, sizeof(struct chat_data), 0666 | IPC_CREAT);
    int semid = semget(SEM_KEY, 1, 0666 | IPC_CREAT);
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

#define SHM_SIZE 1024

int main() {
    key_t key = ftok("chatfile", 65); // Membuat key unik
    int shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT); // Membuat shared memory
    char *shared_memory = (char*) shmat(shmid, NULL, 0);
    char message[100];

    printf("=== CHAT WRITER ===\n");
    printf("Ketik 'exit' untuk keluar.\n\n");

    while (1) {
        printf("Kirim Pesan: ");
        fgets(message, sizeof(message), stdin);
        message[strcspn(message, "\n")] = 0; // Hilangkan newline

        // Tulis pesan ke shared memory
        strcpy(shared_memory, message);

        if (strcmp(message, "exit") == 0) {
            printf("Keluar dari chat.\n");
            break;
        }

        sleep(1); // Jeda agar reader sempat membaca
    }

    shmdt(shared_memory); // Lepas shared memory
    return 0;
}

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

#define SHM_SIZE 1024

int main() {
    key_t key = ftok("chatfile", 65); // Gunakan key yang sama
    int shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    char *shared_memory = (char*) shmat(shmid, NULL, 0);
    char last_message[100] = "";

    printf("=== CHAT READER ===\n");
    printf("Menunggu pesan...\n\n");

    while (1) {
        if (strcmp(last_message, shared_memory) != 0) {
            strcpy(last_message, shared_memory);
            printf("Pesan baru: %s\n", shared_memory);

            if (strcmp(shared_memory, "exit") == 0) {
                printf("Chat berakhir.\n");
                break;
            }
        }
        usleep(200000); // Periksa setiap 0.2 detik
    }

    shmdt(shared_memory);
    shmctl(shmid, IPC_RMID, NULL); // Hapus shared memory
    return 0;
}

```

Kurang lebih hasilnya seperti ini

```
putra@DESKTOP-87VNBLO:~$  
=== CHAT ROOM ===  
putra1> Halo semua!  
yoga2> Hai putra!  
putra1> Bagaimana kabarmu?  
yoga2> Baik, terima kasih|
```

KESIMPULAN PRAKTIKUM 2

Dengan praktikum ini, kita dapat memahami bagaimana sistem operasi mengelola proses mulai dari pembuatan, eksekusi, hingga sinkronisasi antara parent dan child process. Selain itu, diperoleh pemahaman tentang berbagai metode komunikasi antarproses (IPC) seperti Pipe, FIFO, dan Shared Memory. Masing-masing metode memiliki keunggulan dan kelemahan—Pipe sederhana namun terbatas satu arah, FIFO lebih fleksibel, dan Shared Memory paling cepat tetapi memerlukan sinkronisasi tambahan. Secara keseluruhan, praktikum ini memberikan gambaran nyata tentang bagaimana sistem operasi menjalankan multitasking dan komunikasi antarproses secara efisien.